HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO TUẦN 4

“ Xây dựng Website đặt sân thể thao tích hợp AI chatbot và gợi ý thông minh”

Giảng viên hướng dẫn: Kim Ngọc Bách

Họ và tên sinh viên : Bùi Trung Đức

Mã sinh viên : B23DCCN166

Lớp : D23CQCN12-B

Nhóm học phần : 11

Hà Nội, 3/2026

I. Công việc đã thực hiện được trong tuần

1. Kết nối cơ sở dữ liệu:

- Cấu hình database trong file *application.properities:*

```
spring.datasource.url=jdbc:mysql://localhost:3306/sportify?allowPublicKeyRetrieval=true&useSSL=false
spring.datasource.username=TTCS_db


spring.datasource.password=trungduc17


# C\u00c3\u00a1\u00c2\u00ba\u00c2\u00a5u h\u00c3\u0083\u00c2\u00acnh Hibernate
spring.jpa.hibernate.ddl-auto=none
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

spring.data.web.pageable.default-page-size=4
spring.data.web.pageable.max-page-size=100
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
spring.main.allow-circular-references=true
spring.session.store-type=none



spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=phamthihue1669@gmail.com
spring.mail.password=17032005

#spring.mail.properties.mail.smtp.starttls.enable=true
#spring.mail.properties.mail.smpt.auth=true

spring.mail.protocol:smtp
spring.mail.properties.mail.smtp.auth: true
spring.mail.properties.mail.smtp.starttls.enable: true
spring.mail.properties.mail.smtp.ssl.trust: smtp.gmail.com
```

- *jdbc:mysql://localhost:3306/sportify?allowPublicKeyRetrieval=true&useSSL=false:*
+ *jdbc:mysql://*: Giao thức JDBC cho MySQL.
+ *localhost:3306*: Máy chủ database (localhost) và cổng (3306, mặc định của MySQL).
+ *sportify*: Tên database.
+ *allowPublicKeyRetrieval=true*: Cho phép truy xuất khóa công khai từ máy chủ (cần thiết cho một số phiên bản MySQL với SSL).
+ *useSSL=false*: Tắt SSL để tránh lỗi kết nối
- *spring.datasource.username:* Tên đăng nhập vào database trên Mysql
- *spring.datasource.password:* Mật khẩu để đăng nhập trên Mysql
- *spring.jpa.hibernate.ddl-auto = none:* Cấu hình Hibernate không tự động tạo/cập nhật schema database.
- *spring.jpa.properties.hibernate.dialect:* Dialect Hibernate cho MySQL 8, giúp Hibernate tối ưu hóa truy vấn SQL cho phiên bản MySQL cụ thể

2. Backend xử lí phần đăng kí, đăng nhập - Controller nhận POST request, kiểm tra và lưu dữ liệu.

- Trên file *SecurityController.java:*

```java
@Controller
public class SecurityController {
    @Autowired
    UserService userService;

    @Autowired
    UserDAO userDAO;

    @Autowired
    AuthorizedDAO authorizedDAO;

    List<Users> listUser = new ArrayList<>();
    @RequestMapping("/sportify/login")
    public String loginForm(Model model) {
        listUser = userService.findAll();
        model.addAttribute(attributeName: "listUser",listUser);
        return "security/login";
    }
    @RequestMapping("/sportify/signup")
    public String signupForm(Model model) {
        return "security/signup";
    }

    @RequestMapping("/sportify/login/success")
    public String loginSuccess(Model model, HttpSession session, HttpServletRequest request,
            HttpServletResponse response) {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        String username = authentication.getName();
        session.setAttribute("username", username);
        return "redirect:/sportify";
    }
```

```java
@RequestMapping("/sportify/login/error")
public String loginError(Model model, @RequestParam(name = "error", required = false) String error
            model.addAttribute(attributeName: "message", attributeValue: "Thông tin đăng nhập không
            return "security/login";
}

@RequestMapping("/sportify/unauthoried")
public String unauthoried(Model model) {
    model.addAttribute(attributeName: "message", attributeValue: "Không có quyền truy xuất!");
    return "user/error";
}

@RequestMapping("/sportify/logoff/success")
public String logoffSuccess(Model model, HttpSession session) {
    model.addAttribute(attributeName: "message", attributeValue: "Bạn đã đăng xuất!");
    session.removeAttribute("username"); //Xóa Session người dùng đăng nhập
    return "redirect:/sportify";
}

@PostMapping("/sportify/signup/process")
public String processSignup(Model model,HttpServletRequest request, HttpSession session,
        @RequestParam("firstnameSignUp") String firstnameSignUp,
        @RequestParam("lastnameSignUp") String lastnameSignUp,
        @RequestParam("usernameSignUp") String usernameSignUp,
        @RequestParam("passwordSignUp") String passwordSignUp,
        @RequestParam("phoneSignUp") String phoneSignUp,
        @RequestParam("genderSignUp") String genderSignUp,
        @RequestParam("addressSignUp") String addressSignUp,
        @RequestParam("emailSignUp") String emailSignUp) {
```

```java
public class SecurityController {
    public String processSignup(Model model,HttpServletRequest request, HttpSession session,
        Users userOptional = userDAO.findAcc(usernameSignUp);
        // Kiểm tra xem tên đăng nhập đã tồn tại chưa
        if (userOptional !=null ) {
            model.addAttribute(attributeName: "message", attributeValue: "Tài khoản đã tồn tại !");
            return "security/signup";
        }
        boolean isMale = genderSignUp.equals("male");
        // Tạo một đối tượng User mới
        Users newUser = new Users();
        newUser.setLastname(lastnameSignUp);
        newUser.setFirstname(firstnameSignUp);
        newUser.setUsername(usernameSignUp);
        newUser.setPasswords(passwordSignUp);
        newUser.setPhone(phoneSignUp);
        newUser.setEmail(emailSignUp);
        newUser.setAddress(addressSignUp);
        newUser.setStatus(isMale);
        newUser.setGender(gender: true);
        userDAO.save(newUser);
        //Tạo 1 đối tượng Authorized
        Authorized newAuthorized =new Authorized();
        newAuthorized.setUsername(usernameSignUp);
        newAuthorized.setRoleid(roleid: "R03");
        authorizedDAO.save(newAuthorized);
//      System.out.print("Bạn đã đăng kí thành công !");
        return "redirect:/sportify/login"; // hoặc trang bạn muốn
    }


    @CrossOrigin("*")
    @ResponseBody
    @RequestMapping("/rest/security/authentication")
    public Object getAuthentication(HttpSession session) {
        return session.getAttribute("authentication");
    }
```

a. *loginForm* (GET /sportify/login)

- Mục đích: Hiển thị trang đăng nhập.
- Luồng xử lý: Lấy danh sách tất cả user từ *userService.findAll()* (có thể để hiển thị hoặc debug).
- Thêm *listUser* vào model.
- Trả về view security/login (file login.html trong templates).

Ý nghĩa: Chuẩn bị dữ liệu cho trang login, nhưng không xử lý logic đăng nhập (để Spring Security xử lý).

b. *signupForm* (GET /sportify/signup)

- Mục đích: Hiển thị trang đăng ký.
- Luồng xử lý:Không có logic phức tạp, chỉ trả về view security/signup (file signup.html).

Ý nghĩa: Form đăng ký sẽ submit POST đến *processSignup*.

c. *processSignup* (POST /sportify/signup/process)

- Mục đích: Xử lý đăng ký user mới.

- Luồng xử lý: Nhận parameters từ form: firstname, lastname, username, password, phone, gender, address, email.
- Kiểm tra username đã tồn tại: Gọi *userDAO.findAcc(*usernameSignUp) (query DB). Nếu tồn tại, thêm message lỗi vào model và trả về security/signup (hiển thị lại form với lỗi).
- Nếu username chưa tồn tại:Tạo đối tượng Users mới với dữ liệu từ *form.setStatus*(isMale)
- Lưu user vào DB qua *userDAO.save*(newUser).
- Tạo đối tượng Authorized mới với username và roleid = "R03" (role customer mặc định).
- Lưu authorized vào DB qua authorizedDAO.save(newAuthorized).
- Redirect đến /sportify/login (chuyển sang trang login sau khi đăng ký thành công).

Ý nghĩa: Tạo tài khoản mới, gán quyền cơ bản, và chuyển hướng đăng nhập. Không có validation server-side mạnh (chỉ check username trùng).

d. *loginSuccess* (GET /sportify/login/success)

- Mục đích: Xử lý sau khi Spring Security xác thực thành công.
- Luồng xử lý:Lấy Authentication từ S*ecurityContextHolder* (được set bởi Spring Security sau login).
- Trích xuất username từ authentication.
- Lưu username vào *session* (session.setAttribute("username", username)).
- Redirect đến /sportify (trang home).

Ý nghĩa: Lưu thông tin user đã đăng nhập vào session để sử dụng trong các request sau. Đây là callback sau khi login thành công.

5. *loginError* (GET /sportify/login/error)

- Mục đích: Xử lý khi đăng nhập thất bại.
- Luồng xử lý:Nhận parameter error (từ Spring Security).
- Thêm message lỗi vào model: "Thông tin đăng nhập không hợp lệ hoặc tài khoản của bạn đã bị khóa".
- Trả về view *security/login* (hiển thị lại form login với lỗi).

Ý nghĩa: Hiển thị thông báo lỗi cho user, có thể do sai username/password hoặc tài khoản bị khóa (kiểm tra trong CustomUserDetailsService).

6. *logoffSuccess* (GET /sportify/logoff/success)

- Mục đích: Xử lý logout.
- Luồng xử lý:Thêm message "Bạn đã đăng xuất!" vào model.
- Xóa username khỏi session (session.removeAttribute("username")).
- Redirect đến /sportify (trang home).

Ý nghĩa: Dọn dẹp session sau logout, đảm bảo user không còn authenticated.

7. getAuthentication (GET /rest/security/authentication)
   - Mục đích: REST endpoint trả về thông tin authentication (cho frontend hoặc API).
   - Luồng xử lý: Lấy authentication từ session (được set trong SecurityConfig hoặc nơi khác).
   - Trả về object dưới dạng JSON (do @ResponseBody).

Ý nghĩa: Cung cấp thông tin user đã đăng nhập cho các request AJAX hoặc API.

II. Kết quả đạt được

- Kết nối cơ sở dữ liệu MySQL thành công.
- Lưu dữ liệu tài khoản vào database.
- Xây dựng thành công API đăng kí tài khoản, đảm bảo mật khẩu được mã hóa an toàn.
- Xây dựng được luồng Frontend → Backend → Database.

III. Khó khăn gặp phải

- Validate các dữ liệu và xử lí logic khi đăng kí, đăng nhập còn bị thiếu nhiều trường hợp và lỗi.
- Khó khăn khi không đồng bộ được khi lấy dữ liệu từ phần Database.
- Chưa xử lý được Backend phần quên mật khẩu để gửi mã về gmail lấy lại mật khẩu vì còn nhiều lỗi khi kết nối Backend và Database.

IV. Kế hoạch tuần tiếp theo

- Fix các lỗi trong các chức năng xác thực.
- Phát triển chức năng gửi mã OTP về tài khoản gmail để lấy lại mật khẩu
- Hoàn thiện việc đồng bộ dữ liệu giữa Frontend và Backend trong các chức năng xác thực của người dùng.
- Tiếp tục thiết kế phần giao diện đặt sân thể thao.
- Kiểm tra các chức năng đã xây dưng để hệ thống hoạt động mượt mà, không bị lỗi